CSE470: Software Engineering

Software Requirements Specification (SRS)

MediSync- Patient Portal

Prepared By

Name	Student ID
Tahmid Islam	23201054
Sumaiya Islam Aritra	23201162
Afnan Mazumder	24141229
Ramisa Faria	24101084

Project Information

Section	11
Section Faculty	Nazmus Sakib Touhid
GitHub Repository Link	https://github.com/tahmidislam2-star/Medisync
Tech Stack Used	React, Fast API, PostgreSQL

1. Introduction

1.1 Purpose

MediSync is a comprehensive healthcare platform designed for Bangladesh, enabling users to manage medical records, access test reports and prescriptions, find hospitals and doctors, schedule appointments, and receive online consultations. For doctors, MediSync simplifies appointment management, patient history review, messaging, and telehealth services. By connecting patients and healthcare providers in one platform, MediSync makes healthcare more accessible, convenient, and efficient.

1.2 Scope

MediSync minimizes the hassle patients have to go through to find hospital and doctors according to their needs. Meanwhile, Doctors can keep track of patient reports in one go. MediSync aims to centralize healthcare services in Bangladesh by connecting patients, doctors, hospitals, and diagnostic centers through a unified digital platform.

2. Overall Description

2.1 Product Perspective

While most healthcare apps in Bangladesh focus on individual hospitals or telemedicine, MediSync brings the entire healthcare ecosystem together, seamlessly connecting patients, doctors, and hospitals on one unified platform.

2.2 Product High-Level Summary

- *Secure Accounts, Roles, and Privacy*: Users register and log in with Google account, and role-based access control limits each role to what it is permitted to see.
- *Doctor and Hospital Discovery with Transparent Rankings:* Patients search and filter doctors and hospitals by specialty, location, fee, availability, rating, and distance, and view detailed profile pages.
- *Appointments, Payments, and Telehealth:* Patients book real-time slots, pay consultation fees online, and reschedule or cancel.
- *Lab Tests and Results Management*: Doctors order lab tests, and each test is tracked.

2.3 Users and Characteristics

- Patient: Search, book, view reports, message, review.
- Doctor: Manage schedule, view patient history, reply to messages, run telehealth sessions.
- Hospital Admin: Upload test results, manage the hospital profile, verify doctors and hospitals, departments, and lab turnaround data.
- System Admin: Moderate reviews, manage the platform.

2.4 Operating Environment

MediSync is a web app. It runs in any modern desktop browser.

2.5 Design and Implementation Constraints

(a) Primary Tech Stack

Frontend: React with Tailwind CSS

Backend: Fast API (Python)

Database: PostgreSQL, accessed through SQL Alchemy

Real-time: WebSocket for chat and notifications

Integrations: bKash (payments), Jitsi (telehealth video), Google Maps (hospital distance), and a PDF library for reports and prescriptions.

(b) MVC Framework and Structure:

Model (app/models/, app/schemas/): SQL Alchemy models (User, Patient, Doctor, Hospital, Appointment, Lab Test, Lab Report, Message, Prescription, Review, Payment).

View (frontend/src/): React pages and components, such as the patient dashboard, doctor search, booking flow, report viewer, chat, and admin panel.

Controller (app/routers/, app/services/): Routers (AuthController, DoctorController, HospitalController, AppointmentController, LabController, MessageController, TelehealthController) handle requests, and the services layer holds business logic such as slot holding, payment verification, and ranking.

(c) AI-backed feature: Not Applicable.

3. System Architecture

3.1 Architectural Overview

To Be Implemented

3.2 Technology Stack

Frontend: React with Tailwind CSS, using React Router for navigation and Axios for API calls.

Backend: Fast API (Python) with SQL Alchemy as the ORM.

Database: PostgreSQL for relational data such as users, appointments, payments, and lab reports.

Real-time: WebSocket (Fast API) for chat and notifications.

Authentication: OTP login via Google and role-based access control.

3.3 Class Diagram

To Be Implemented

4. Requirements and Features

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1	User Registration	Patients can login/sign up with Google.	Patient			
R2	Role Based Access	Hospital admins register doctors to their hospitals.	Hospital Admin			
R3	Patient Profile and Medical History	Patients can view and change their personal information. They can also view their previous medical reports such as prescriptions and test results.	Patient			
R4	Doctor Search	Users can search doctor by specialty, location, fee, and availability.	Patient/ Viewer			
R5	Hospital Filtering	Users can filter hospitals by ranking, department, cost, and distance.	Patient/ Viewer			
R6	Doctor and Hospital Profile Page	Doctors and hospital can manage their profile by editing/ adding their information.	Doctor and Hospital Admin			
R7	Appointment Booking with Real-time Slots	Patients can book their preferred slots for doctor appointment	Patient			
R8	Reschedule and Cancel Appointments	Patients can reschedule or cancel their appointments if needed.	Patient			

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R9	Real-time and Video Calling Between Doctors and Patients	Patients can consult with their preferred doctors online by text messages or video calls.	Patient			
R10	Online Payment for Telemedicine Service	Patients can pay for telemedicine services and book slots.	Patient			
R11	Lab Test Ordering and Tracking	Patients can track their lab tests (Ordered → Processing → Ready) and also book lab tests in their preferred hospital.	Patient			
R12	Secure Lab Result Downloading	Patients can download lab report as pdf file and save it in their local device.	Patient			
R13	E-Prescription generation	Doctors can generate prescriptions for individual patients.	Doctor			
R14	Report Comparison Overtime	Patients can compare their current lab reports with their past reports.	Patient			
R15	User Removal and Workflow	Hospital admins can ban/ remove specific doctors or patients.	Hospital Admin			
R16	Hospital Service Information	Verified users can rate hospitals and doctors.	Patient			
R17	Hospital and Doctor Rating	Verified users can rate hospitals and doctors.	Patient			
R18	Doctor Profile view	Doctors can view and track their patients.	Doctor			
R19	Review Moderation	System Admin can manage reviews.	System Admin			
R20	Hospital Access Control	System manager can register a hospital and add hospital admin.	System Admin	…	…	…

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20

Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Responsive Interface.
Hardware Interface	Should run in any common web browser.

6. Non-Functional Requirements (Optional)

Category	Requirement
Performance	User messages are delivered in real-time.
Security	Sensitive health data is encrypted.

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1			
Sprint 2			
Sprint 3			
Sprint 4			

7.1 Deviations and Lessons Learned

---

8. Testing Summary

---

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)

10. Individual Contribution Summary

---

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	
At least 20 features are listed and each is correctly counted per the granularity rule	
No forbidden framework (e.g. Django, Flask as primary backend) was used	
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	
The AI Usage Disclosure table (Section 10) is complete and honest	
I can personally explain and defend every feature listed in this document during the viva	